Simulador de Paginação de Memória Virtual: planejamento do trabalho

Title: A Virtual Memory Paging Simulator: work plan

Pedro Luiz Pereira Zampar[1], **Pedro Augusto dos Santos Watermann**[1]

[1]Departamento de Informática – Universidade Estadual de Maringá (UEM)
Av. Colombo, 5790 – 87.020-900 – Maringá – PR – Brasil

ra145117@uem.br, ra145121@uem.br

Resumo. *Este documento apresenta o planejamento de um simulador do mecanismo de substituição de páginas empregado pelo gerenciador de memória virtual de um sistema operacional. A partir de uma sequência de referências a páginas e de um número fixo de quadros físicos, o simulador contabiliza as faltas de página ocorridas e reproduz, referência a referência, o estado resultante dos quadros. A mesma carga de trabalho é então submetida a cinco políticas de substituição — FIFO, LRU, Ótimo, Segunda Chance e LFU — , de modo que seus comportamentos possam ser contrastados à medida que se varia a memória disponível. Descrevemos a organização pretendida dos módulos, as assinaturas que definem a interface entre eles, o comportamento esperado do programa, os experimentos que pretendemos conduzir — entre eles a reprodução da Anomalia de Belády — e a divisão do trabalho em etapas.*
Palavras-Chave *Sistemas Operacionais, Memória Virtual, Paginação, Substituição de Páginas, Anomalia de Belády.*

1. Objeto do trabalho

O trabalho consiste em construir um programa que reproduza a forma como o sistema operacional administra a memória de um processo cujo espaço de endereçamento não cabe integralmente na memória física. O simulador recebe como entrada a sequência de páginas referenciadas por esse processo e a quantidade de quadros disponíveis; a partir desses dois dados, contabiliza quantas vezes as páginas requisitadas não se encontravam residentes — as faltas de página — e registra o passo a passo da execução.

O propósito é submeter a mesma sequência a cinco algoritmos de substituição distintos e observar qual é o mais adequado em cada cenário, medindo a quantidade de faltas em cada algoritmo, conforme as entradas e a memória disponível são variadas. Este documento registra o entendimento que temos do problema e a estruturação pretendida para a solução, ressalvado que parte das decisões poderá ser revista conforme o conteúdo for tratado em aula.

1.1. Entrada e saída

- **Entrada:** um arquivo contendo a sequência de páginas referenciadas (por exemplo, `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`) e o número de quadros da memória física.
- **Saída:** o total de faltas de página, o traço da execução exibindo o estado dos quadros a cada referência e um gráfico relacionando as faltas ao número de quadros.

2. Fundamentação conceitual

A memória virtual é dividida em blocos de tamanho fixo denominados páginas, e a memória física, em blocos de mesmo tamanho denominados quadros [Silberschatz et al. 2018]. Quando o processo referencia uma página que não está carregada em nenhum quadro, ocorre uma falta de página: havendo quadro livre, basta carregá-la; estando a memória ocupada, é preciso eleger uma página residente para ser removida, e é essa decisão que caracteriza cada algoritmo de substituição [Tanenbaum e Bos 2015]. A Tabela 1 sintetiza os critérios das cinco políticas que implementaremos.

Tabela 1. Critério de escolha da página vítima em cada algoritmo.

Algoritmo	Página eleita para substituição
FIFO	A residente há mais tempo, independentemente de estar ou não em uso
LRU	A que permaneceu por mais tempo sem ser referenciada
Ótimo	A que demorará mais para ser referenciada novamente (pressupõe conhecer as páginas que virão a seguir)
Segunda Chance	Como o FIFO, porém preservando páginas referenciadas recentemente com um bit de referência
LFU	A que acumulou o menor número de referências

Anomalia de Belády. A intuição sugere que ampliar a memória física deva reduzir o número de faltas de página. Há, contudo, sequências para as quais essa expectativa não se confirma e o FIFO passa a apresentar desempenho pior com mais quadros do que com menos [Belády et al. 1969]. Iremos reproduzir experimentalmente esse fenômeno e evidenciar que ele não se manifesta no LRU nem no Ótimo. [Arpaci-Dusseau e Arpaci-Dusseau 2018].

3. Organização do projeto

Cada algoritmo fica em seu próprio módulo, e todos usam a mesma assinatura de função. Isso permite trocar de política ou mudar o número de quadros sem precisar alterar o resto do programa.

```
paging-simulator/
|-- src/
|   |-- main.py            # le os parametros e despacha a simulacao
|   |-- simulador.py       # tipos compartilhados por todos os modulos
|   |-- sequencia.py       # leitura do arquivo de entrada
|   |-- traco.py           # impressao do traco e do resumo
|   |-- fifo.py            # algoritmo FIFO
|   |-- lru.py             # algoritmo LRU
|   |-- otimo.py           # algoritmo Otimo
|   |-- segunda_chance.py  # algoritmo Segunda Chance
|   `-- lfu.py             # algoritmo LFU
|-- entradas/              # sequencias de teste
|-- analise.py             # geracao dos graficos a partir do CSV
|-- Makefile               # alvos de execucao, testes e graficos
`-- RELATORIO.pdf
```

3.1. Escolha da linguagem

Optamos por implementar o simulador em Python, por familiaridade prévia com a linguagem, o que agiliza o desenvolvimento, e pela padronização com os arquivos de

análise, que já seriam escritos em Python para uso do *matplotlib*. O projeto seguirá com `Makefile` funcional e execução em ambiente Linux, conforme exigido.

3.2. Ambiente de testes

Os experimentos serão conduzidos em um notebook Acer Nitro V15. O programa não roda diretamente sobre o Windows: a execução e os testes são feitos em um ambiente Ubuntu Linux hospedado via WSL2 (*Windows Subsystem for Linux*, versão 2). Como o simulador opera apenas sobre listas de inteiros pequenas, os recursos relevantes são a CPU e a memória principal:

- **CPU:** AMD Ryzen 7 7735HS, 8 núcleos / 16 *threads*, 3,2 GHz (até 4,75 GHz); cache L1 de 64 KB e L2 de 512 KB por núcleo, L3 compartilhada de 16 MB
- **Memória RAM:** 16 GB DDR5-4800
- **Distribuição:** Ubuntu 26.04.1 LTS (codinome *Resolute*)
- **Kernel:** Linux 6.18.33.2-microsoft-standard-WSL2
- **Interpretador:** Python 3.14.4
- **Virtualização:** WSL2 sobre Hyper-V, com memória dinamicamente alocada à VM (*dynamic memory allocation*) a partir dos 16 GB de RAM física disponíveis

4. Estruturas de dados e assinaturas

4.1. Tipos compartilhados

Duas estruturas atravessam todo o programa, ambas definidas em `src/simulador.py`. A primeira representa a sequência de páginas lida do arquivo de entrada; a segunda condensa o produto de uma simulação, para um dado algoritmo e um dado número de quadros:

```
from typing import TextIO

Traco = TextIO | None            # destino do traco (None no modo em lote)

@dataclass
class Sequencia:
    paginas: list[int]           # paginas na ordem em que foram referenciadas

@dataclass
class Resultado:
    algoritmo: str               # nome da politica simulada
    n_quadros: int               # quadros disponiveis na memoria fisica
    faltas: int                  # total de faltas de pagina
    acertos: int                 # referencias a paginas ja residentes
```

Uma vez que os cinco algoritmos devolvem esse mesmo tipo, os resultados podem ser agregados em uma única tabela e exportados para CSV sem qualquer tratamento particular a cada política. O apelido `Traco` indica onde o traço é impresso; o valor `None` o suprime.

4.2. Funções auxiliares

```
# src/sequencia.py
def ler_sequencia(caminho: str) -> Sequencia

# src/simulador.py
def buscar_quadro(quadros: list[int | None], pagina: int) -> int
```

```
# src/traco.py
def traco_linha(saida: TextIO, algoritmo: str, pagina: int,
                quadros: list[int | None], houve_falta: bool,
                vitima: int | None) -> None
def traco_resumo(saida: TextIO, resultado: Resultado) -> None
```

A `ler_sequencia` converte o conteúdo do arquivo em uma lista de inteiros, aceitando os números separados por espaços ou por quebras de linha. A `buscar_quadro` localiza uma página entre os quadros e devolve sua posição, ou -1 caso ela não esteja residente: é ela que determina se houve acerto ou falta. As duas últimas cuidam da saída — uma linha de traço por referência e o resumo final —, mantida isolada dos algoritmos para não replicar esse código cinco vezes.

4.3. Os cinco algoritmos

Todas as funções de simulação recebem os mesmos parâmetros e devolvem o mesmo tipo, variando exclusivamente na regra de escolha da página vítima:

```
# um modulo por algoritmo: src/fifo.py, lru.py, otimo.py, segunda_chance.py, lfu.py
def simular_fifo(seq: Sequencia, n_quadros: int, traco: Traco) -> Resultado
def simular_lru(seq: Sequencia, n_quadros: int, traco: Traco) -> Resultado
def simular_otimo(seq: Sequencia, n_quadros: int, traco: Traco) -> Resultado
def simular_segunda_chance(seq: Sequencia, n_quadros: int, traco: Traco) -> Resultado
def simular_lfu(seq: Sequencia, n_quadros: int, traco: Traco) -> Resultado

# src/otimo.py: posicao da proxima referencia a pagina depois de posicao,
# ou len(seq.paginas) se ela nao voltar a ser referenciada
def proximo_uso(seq: Sequencia, posicao: int, pagina: int) -> int
```

- **FIFO:** a ideia é organizar as páginas na ordem em que chegaram e sempre remover a mais antiga, como uma fila. Basta guardar um índice indicando qual quadro será o próximo a sair, e avançar esse índice sempre que uma substituição acontecer. Se a página referenciada já estiver na memória, nada muda nessa fila — e é exatamente por ignorar se a página foi usada recentemente que o FIFO pode sofrer da Anomalia de Belády (quando aumentar o número de quadros piora o desempenho, ao invés de melhorar).
- **LRU:** o objetivo é sempre remover a página que não é usada há mais tempo. Para isso, mantemos um contador que aumenta a cada referência, e guardamos, para cada página na memória, o valor desse contador no momento do seu último acesso. Quando é preciso substituir uma página, escolhemos a que tem o menor valor guardado (ou seja, a que foi usada há mais tempo). Isso é mais eficiente do que revisar toda a sequência de referências a cada vez: usando um dicionário para guardar esses valores, atualizar o acesso de uma página é rápido (tempo constante), e encontrar a vítima exige apenas percorrer os quadros ocupados.
- **Ótimo:** como já temos a sequência inteira de referências disponível, podemos "olhar para o futuro"e verificar quando cada página na memória será usada novamente. A página escolhida para sair é a que vai demorar mais para ser referenciada de novo ou uma que nunca mais será usada. Uma função auxiliar, `proximo_uso`, fica responsável por descobrir a posição da próxima referência a cada página.
- **Segunda Chance:** é uma versão melhorada do FIFO. Cada página ganha um "bit de referência", que é ativado toda vez que ela é acessada. Na hora de substituir, olhamos para a página mais antiga: se o bit dela estiver ativado, isso significa que ela foi usada recentemente, então damos a ela uma segunda chance desativando o bit e passando para

a próxima página mais antiga, ao invés de removê-la. Se todas as páginas tiverem o bit ativado, o algoritmo acaba se comportando exatamente como o FIFO.

- **LFU:** aqui contamos quantas vezes cada página foi referenciada, e removemos a que tem o menor número de acessos. Em caso de empate entre páginas com a mesma contagem, usamos o mesmo critério do FIFO: removemos a que está na memória há mais tempo.

4.4. Programa principal e análise

```
# src/main.py
def main(argv: list[str]) -> int

# analise.py
def plotar_faltas_por_quadros(dados: list[Resultado], saida: str) -> None
def detectar_anomalia_belady(dados: list[Resultado]) -> list[tuple[str, int]]
```

A `main` interpreta os parâmetros da linha de comando, carrega a sequência, seleciona a função de simulação correspondente e apresenta o resultado; concentrar essa seleção em um único ponto evita que condicionais relativas ao algoritmo se espalhem pelo programa. A `plotar_faltas_por_quadros` constrói o gráfico com o número de quadros no eixo x e as faltas no eixo y, com uma curva por algoritmo, e o grava no arquivo `saida`; a `detectar_anomalia_belady` confronta quantidades consecutivas de quadros e devolve os pares (algoritmo, número de quadros) em que as faltas aumentaram (ocorrências da anomalia).

5. Comportamento esperado do programa

A execução se dará por linha de comando, informando-se o arquivo de entrada, o número de quadros e o algoritmo desejado. A saída prevista é a seguinte:

```
$ python3 -m ./src/main.py --entrada entradas/sequencia1.txt --quadros 3 --algoritmo fifo

Ref: 7  Quadros: [7,-,-]   FALTA
Ref: 0  Quadros: [7,0,-]   FALTA
Ref: 1  Quadros: [7,0,1]   FALTA
Ref: 2  Quadros: [2,0,1]   FALTA (substitui 7 - FIFO)
Ref: 0  Quadros: [2,0,1]   ACERTO
...
Algoritmo: FIFO | Quadros: 3 | Referencias: 20 | Faltas: 15
```

Haverá ainda um modo de execução em lote, que percorre todos os algoritmos com o número de quadros variando de 1 até um limite informado. Nesse modo o traço é suprimido e os resultados são gravados em um arquivo CSV, consumido em seguida pelo script de análise para a construção do gráfico comparativo.

6. Plano de testes e experimentos

- Executar a sequência de exemplo do enunciado do trabalho com 3 e 4 quadros e conferir manualmente o resultado, assegurando a correção da contagem.
- Submeter a sequência `1 2 3 4 1 2 5 1 2 3 4 5` ao FIFO com 3 e, em seguida, com 4 quadros, cenário em que esperamos observar a Anomalia de Belády.
- Repetir esse experimento com o LRU e com o Ótimo, evidenciando que neles a anomalia não se manifesta.
- Gerar uma sequência mais extensa, com reincidência periódica de páginas, para contrastar o Segunda Chance com o FIFO puro e identificar em que situações os dois divergem.

- Variar o número de quadros de 1 a 30 em todos os algoritmos e consolidar os resultados no gráfico final.

Referências

Arpaci-Dusseau, R. H. e Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.00 edition.

Belády, L. A., Nelson, R. A., e Shedler, G. S. (1969). An anomaly in space-time characteristics of certain programs running in a paging machine. *Communications of the ACM*, 12(6):349–353.

Silberschatz, A., Galvin, P. B., e Gagne, G. (2018). *Operating System Concepts*. Wiley, 10 edition.

Tanenbaum, A. S. e Bos, H. (2015). *Modern Operating Systems*. Pearson, 4 edition.